Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по лабораторной работе №4 «Работа с файлами и структуры данных на основе указателей»

по дисциплине «Программирование»

Вариант 2-4-18-3

Факультет: Безопасность информационных технологий

Группа: N3149

Выполнила: Беленькая Полина Александровна

Подпись:

Преподаватель: Гирик Алексей Валерьевич

Санкт-Петербург 2025

Задание

Разработать на языке C для ОС Linux программу, которая читает из файла заданного формата состояние заданной структуры данных, выполняет манипуляции со структурой в памяти, считывая команды из стандартного потока ввода, и сохраняет результат в файл.

Также для проверки программы написать скрипт на языке Python, который позволяет создать файл заданного формата со случайными данными.

Тип списка: двусвязный список. Данные для поиска: номер кредитной карты. Команда: reverse.

Содержимое файла: в файле последовательно располагаются размеры строк и сами строки. Порядок следования строк в файле соответствует порядку следования строк в списке.

Makefile

```
  GNU nano 7.2
.PHONY: all clean

APP=prg4pabN3149
CFLAGS=-Wall -Wextra -Werror -g

all: $(APP)

$(APP): $(APP).c
        gcc -o $(APP) $(CFLAGS) $(APP).c

clean:
        rm $(APP)
```

Примеры работы программ на различных исходных данных:

```
polina@DESKTOP-QBMU2N6:~$ ./prg4pabN3149 -v
Полина Александровна Беленькая, гр. №3149
Вариант: 2-4-18-3
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg4pabN3149
Ошибка: не задано имя файла.
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg4pabN3149 /no/such/path.txt
Ошибка: не удалось создать файл.
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg4pabN3149 ty.txt
magic
Ошибка: неподдерживаемая команда: 'magic'.
```

```
polina@DESKTOP-QBMU2N6:~$ python3 prg4pabN3149_gen.py -n 5 ty.txt
b'\x13\x00\x00\x00' 6411-0623-2097-6004
b'\x13\x00\x00\x00' 0023-4706-9395-4172
b'\x13\x00\x00\x00' 3896-3311-9334-7515
b'\x13\x00\x00\x00' 0963-6539-5077-3052
b'\x13\x00\x00\x00' 4749-4483-3798-0477
```

```
polina@DESKTOP-QBMU2N6:~$ python3 prg4pabN3149_gen.py -v
Полина Александровна Беленькая, гр. N3149
Вариант 2-4-18-3
```

```
polina@DESKTOP-QBMU2N6:~$ ./prg4pabN3149 ty.txt
print_list
[0]  6411-0623-2097-6004
[1]  0023-4706-9395-4172
[2]  3896-3311-9334-7515
[3]  0963-6539-5077-3052
[4]  4749-4483-3798-0477
pop_back
print_list
[0]  6411-0623-2097-6004
[1]  0023-4706-9395-4172
[2]  3896-3311-9334-7515
[3]  0963-6539-5077-3052
pop_front
print_list
[0]  0023-4706-9395-4172
[1]  3896-3311-9334-7515
[2]  0963-6539-5077-3052
reverse
print_list
[0]  0963-6539-5077-3052
[1]  3896-3311-9334-7515
[2]  0023-4706-9395-4172
push_front 4749-4483-3798-0477
push_back 6411-0623-2097-6004
print_list
[0]  4749-4483-3798-0477
[1]  0963-6539-5077-3052
[2]  3896-3311-9334-7515
[3]  0023-4706-9395-4172
[4]  6411-0623-2097-6004
dump
(nil) 0x5584cec3b630 0x5584cec3b5d0 4749-4483-3798-0477
0x5584cec3b630 0x5584cec3b5d0 0x5584cec3b570 0963-6539-5077-3052
0x5584cec3b5d0 0x5584cec3b570 0x5584cec3b510 3896-3311-9334-7515
0x5584cec3b570 0x5584cec3b510 0x5584cec3bf30 0023-4706-9395-4172
0x5584cec3b510 0x5584cec3bf30 (nil) 6411-0623-2097-6004
```

Исходный код программы:

Текст кода на С:

```
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>

//Структура Двусвязного списка
typedef struct list_t{
	struct list_item_t *head,
	*tail;
	long long int count;
}list_t;

//Структура элемента двусвязного списка
typedef struct list_item_t{
	char* str;
	struct list_item_t	*prev,
	*next;
}list_item_t;

list_t list = {
	.head = NULL,
	.tail = NULL,
	.count = 0
};

//Проверка расширения
int has_txt_extension(const char *filename){
```

```
	const char *dot = strrchr(filename, '.');
	if (!dot || dot == filename)
		return 0;
	return strcmp(dot, ".txt") == 0;
}

bool modified = false; //Проверка наличия изменений

//Определеление всех функций
list_item_t *create_new_list(char *s);
void file_input(FILE *fin);
void push_front(list_t *list, list_item_t *new_str);
void push_back(list_t *list, list_item_t *new_str);
void pop_front(list_t *list);
void pop_back(list_t *list);
void print_list(list_t *list);
void define_command(char *command_name, char **cards, long long int size);
bool is_valid_card(char *str);
void dump_list(list_t *list, char *filename);
void reverse_list(list_t *list);

int main(int argc, char *argv[]){
	FILE *fin = stdin;
	if (argc == 2) {
		if (strcmp(argv[1], "-v") == 0) {
			printf("Полина Александровна Беленькая, гр. №3149\nВариант: 2-4-18-3\n");
			return EXIT_SUCCESS;
		} else{
			if (!has_txt_extension(argv[1])){
				fprintf(stderr, "Ошибка: некорректный формат файла.\n");
				return EXIT_FAILURE;
			}
			fin = fopen(argv[1],"r+");
			if (!fin) {
				fin = fopen(argv[1],"w+");
				if (!fin){
					fprintf(stderr, "Ошибка: не удалось создать файл.\n");
					return EXIT_FAILURE;
				}
				fclose(fin);
				fin = fopen(argv[1],"r+");
			}
			fseek(fin, 0, SEEK_END);
			long file_size = ftell(fin);
			fseek(fin, 0, SEEK_SET);

			if (file_size > 0){
				file_input(fin);
			} else {
```

```
                        printf("Файл пуст. Создан пустой список.\n");
                }
        }
} else {
        printf("Ошибка: не задано имя файла.\n");
        return EXIT_FAILURE;
}

char *str = NULL, *command = NULL;
size_t str_size = 0;
fin = stdin;
char **cards = NULL;
long long int cnt_word = 0;
while (getline(&str, &str_size, stdin) != -1) {
        int j = 0;
        while (str[j] == ' ') j++;
        command = &str[j];
        long long int len =  strlen(command), first_letter = 0, last_letter = 0;
        cnt_word = 0;

        cards = NULL;

        char *command_name = NULL;

        for (long long int i = 0; i <= len; i++){
                if (command[i] == ' ' || command[i] == '\n' || command[i] == '\0'){
                        if (i==first_letter){
                                first_letter = i+ 1;
                                continue;
                        }
                        last_letter = i - 1;
                        char *word = malloc((last_letter-first_letter+2) * sizeof(*word));
                        if (!word){
                                printf("Ошибка: невозможно выделить память.\n");
                                return EXIT_FAILURE;
                        }
                        strncpy(word, &command[first_letter], last_letter - first_letter + 1);
                        word[last_letter - first_letter + 1] = '\0';
                        if (cnt_word == 0){
                                command_name = word;
                        } else{
                                char **temp = realloc(cards, cnt_word * sizeof(*cards));
                                if (!temp) {
                                        printf("Ошибка: невозможно перераспределить память.");
                                        free(word);
                                        return EXIT_FAILURE;
                                }
                                cards = temp;
                                cards[cnt_word - 1] = word;
```

```
                                        }
                                        cnt_word++;
                                        first_letter = i + 1;
                                }
                        }
                        if (command_name) {
                                define_command(command_name, cards, cnt_word -1);
                                free(command_name);
                        }

                        if (cards) {
                                for (int i; i < cnt_word - 1; i++ ) {
                                        free(cards[i]);
                                }
                                free(cards);
                                cards = NULL;
                        }
                }
                free(str);
                fclose(fin);
                if (modified){
                        FILE *fout = fopen(argv[1], "wb");
                        if (!fout) {
                                printf("Ошибка: не удалосьоткрыть файл для записи.\n");
                        } else{
                                for (list_item_t *node = list.head; node != NULL; node = node->next) {
            uint32_t str_len = strlen(node->str);
            if (fwrite(&str_len, sizeof(uint32_t), 1, fout) != 1) {
                printf("Ошибка записи длины строки.\n");
                break;
            }
            if (fwrite(node->str, sizeof(char), str_len, fout) != str_len) {
                printf("Ошибка записи строки.\n");
                break;
            }
        }
        fclose(fout);
    }
}


    return EXIT_SUCCESS;
}
list_item_t *create_new_list (char *s) {
    list_item_t *new_str = malloc(sizeof(*new_str));
    if (!new_str) {
        printf("Ошибка: невозможно выделить память для узла.\n");
        return NULL;
    }
```

```
    size_t len = strlen(s);
    new_str->str = malloc(len + 1);
    if (!new_str->str) {
        printf("Ошибка: невозможно выделить память для строки.\n");
        free(new_str);
        return NULL;
    }

    strcpy(new_str->str, s);
    new_str->next = new_str->prev = NULL;
    return new_str;
}

// Считывание списка с файла
void file_input(FILE *fin) {
    fseek(fin, 0, SEEK_END);
    long file_size = ftell(fin);
    fseek(fin, 0, SEEK_SET);

    while (ftell(fin) < file_size) {
        uint32_t str_len;
        if (fread(&str_len, sizeof(uint32_t), 1, fin) != 1) {
            printf("Ошибка: не удалось считать длину строки.\n");
            break;
        }

        char *s = malloc(str_len + 1);
        if (!s) {
            printf("Ошибка: невозможно выделить память.\n");
            abort();
        }

        if (fread(s, 1, str_len, fin) != str_len) {
            printf("Ошибка: не удалось прочитать строку.\n");
            free(s);
            break;
        }

        s[str_len] = '\0';
        if (is_valid_card(s)) {
                                    push_back(&list, create_new_list(s));
                            } else {
            free(s);
        }
    }
    modified = false; // Сбрасываем флаг, так как это не изменение списка
}
```

```
// Вставка элемента в начало списка
void push_front(list_t *list, list_item_t *new_str) {
    if (list->head) {
        new_str->next = list->head;
        list->head->prev = new_str;
        list->head = new_str;
    } else {
        list->head = list->tail = new_str;
    }
    list->count++;
    modified = true; // Устанавливаем флаг изменений
}

// Вставка элемента в конец списка
void push_back (list_t *list, list_item_t *new_str) {
    if (list->head) {
        new_str->prev = list->tail;
        list->tail->next = new_str;
        list->tail = new_str;
    }
    else {
        list->head = list->tail = new_str;
    }
            list->count++;
            modified = true;
}

// Удаление элемента из начала списка
void pop_front(list_t *list) {
    if (!list->head) {
        return;
    }

    list_item_t *first_node = list->head;
    list->head = list->head->next;

    if (list->head) {
        list->head->prev = NULL;
    } else {
        list->tail = NULL;
    }

    list->count--;
    free(first_node->str);
    free(first_node);
    modified = true;
}
```

```
void pop_back(list_t *list) {
    if (!list->tail) {
        return;
    }

    list_item_t *last_node = list->tail;
    list->tail = list->tail->prev;

    if (list->tail) {
        list->tail->next = NULL;
    } else {
        list->head = NULL;
    }

    list->count--;
    free(last_node->str);      // Освобождаем строку
    free(last_node);           // Освобождаем сам узел
    modified = true;
}

// Вывод списка
void print_list (list_t *list) {
    list_item_t *curr_node = list->head;
    unsigned long i = 0;
    while(curr_node) {
        printf("[%lu]  %s\n", i, curr_node->str);
        curr_node = curr_node->next;
        i++;
    }
}

// Распознавание команды и выполнение ее
void define_command (char *command_name, char **cards, long long int size) {
    if (strcmp(command_name, "push_front") == 0) {
        for(long long int i = size - 1; i >= 0; i--) {
            if (is_valid_card(cards[i])) {
                push_front(&list, create_new_list(cards[i]));
            }
            else {
                printf ("Ошибка: карта %s нек.\n", cards[i]);
            }
         }
        return;
    }
    if (strcmp(command_name, "push_back") == 0) {
        for (long long int i = 0; i < size; i++) {
            if (is_valid_card(cards[i])) {
                push_back(&list, create_new_list(cards[i]));
            }
```

```
            else {
                printf ("Ошибка: карта с номером %s не была добавлена в спискок.\nДанная карта не прошла проверку корректности.\n",
cards[i]);
            }
        }
        return;
    }
    if (strcmp(command_name, "pop_front") == 0) {
        if (size != 0) {
            printf("Ошибка: у данной команды нет аргументов.\n");
            return;
        }
        pop_front(&list);
        return;
    }
    if (strcmp(command_name, "pop_back") == 0) {
        if (size != 0) {
            printf("Ошибка: у данной команды нет аргументов.\n");
            return;
        }
        pop_back(&list);
        return;
    }
    if (strcmp(command_name, "print_list") == 0) {
        if (size != 0) {
            printf("Ошибка: у данной команды нет аргументов.\n");
            return;
        }
        print_list(&list);
        return;
    }
    if (strcmp(command_name, "dump") == 0) {
    if (size > 1) {
        printf("Ошибка: данная команда принимает только 1 аргумент.\n");
        return;
    }
    if (size == 1) {
        dump_list(&list, cards[0]);  // Имя файла указано
    } else {
        dump_list(&list, NULL);             // Имя файла не указано
    }
    return;
}
                if (strcmp(command_name, "reverse") == 0) {
        if (size != 0) {
            printf("Ошибка: у данной команды нет аргументов.\n");
            return;
        }
        reverse_list(&list);
```

```
        return;
    }
    printf("Ошибка: неподдерживаемая команда: '%s'.\n", command_name);
    return;
}

// Проверка по алгоритму Луна
bool is_valid_card (char *str) {
    long long int len = strlen(str), digit = 0, sum = 0;
    bool position = false;
    if (len != 19 || str[4] != '-' || str[9] != '-' || str[14] != '-'){
        return false;
    }
    for (int i = 0; i < len; i++) {
        if (i != 4 && i != 9 && i != 14 && (str[i] < '0' || str[i] > '9')) {
            return false;
        }
    }
    for (int i = len-1; i >= 0; i--) {
        if (str[i] == '-') {
            continue;
        }
        digit = str[i] - '0';
        if (position) {
            digit *= 2;
            if (digit > 9) {
                digit -= 9;
            }
        }
        sum += digit;
        position = !position;
    }
    return sum % 10 == 0;
}

// Дамп списка в файл
void dump_list(list_t *list, char *filename) {
    // Определяем, куда выводить данные: в файл или в stdout
    FILE *fout = NULL;
    if (filename != NULL) {
        // Если имя файла указано, пытаемся открыть его для записи
        fout = fopen(filename, "w+");
        if (!fout) {
            // Если не удалось открыть файл, пробуем открыть его в режиме добавления
            fout = fopen(filename, "a+");
                                    printf("Файл не найден, создан новый.");
        }
        if (!fout) {
            // Если ни один из режимов не сработал, выводим ошибку и завершаем функцию
```

```
            printf("Ошибка: не удалось создать или открыть файл '%s'.\n", filename);
            return;
        }
    } else {
        // Если имя файла не указано, используем стандартный поток вывода
        fout = stdout;
    }

    // Проходим по всем элементам списка и выводим их
    for (list_item_t *i = list->head; i != NULL; i = i->next) {
        fprintf(fout, "%p %p %p %s\n",
                (void *)i->prev,  // Адрес предыдущего элемента
                (void *)i,        // Адрес текущего элемента
                (void *)i->next,  // Адрес следующего элемента
                i->str);          // Строка
    }

    // Закрываем файл, если он был открыт (не stdout)
    if (filename != NULL && fout != NULL) {
        fclose(fout);
    }
}

void reverse_list(list_t *list) {
    if (!list || !list->head) {
        return;
    }

    list_item_t *current = list->head;
    list_item_t *temp = NULL;

    while (current) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }

    temp = list->head;
    list->head = list->tail;
    list->tail = temp;

    modified = true; // Устанавливаем флаг изменений
}
```

Текст скрипта на Python:

```
#!/usr/bin/env python3
import random
import sys
```

```
#Вычисление контрольной суммы по алгоритму Луна
def luhn_checksum(card_number):
    digits = [int(d) for d in card_number if d.isdigit()]
    checksum = 0
    parity = len(digits) % 2
    for i, digit in enumerate(digits):
        if i % 2 == parity:
            digit *= 2
            if digit > 9:
                digit -= 9
        checksum += digit
    return checksum % 10

#Генерация случайного номера карты в формате XXXX-XXXX-XXXX-XXXX
def generate_card_number():
    random_part = ''.join(str(random.randint(0, 9)) for _ in range(15))
    checksum = luhn_checksum(random_part + '0')
    check_digit = (10 - checksum) % 10
    full_card_number = random_part + str(check_digit)
    formatted_card_number = '-'.join([full_card_number[i:i+4] for i in range(0, 16,
4)])
    return formatted_card_number

def gener(fin, n):
    ind_list = list(range(n))
    random.shuffle(ind_list)  #Перемешиваем номера строк

    for i in range(n):
        card_number = generate_card_number()
        card_byte = card_number.encode('utf-8')
        #Записываем длину номера карты и сам номер
        len_byte = len(card_byte).to_bytes(4, byteorder='little')
        fin.write(len_byte)
        fin.write(card_byte)
        print(len_byte, card_number)

def main():
    f_name = None
    n = -1
    if len(sys.argv) == 1:
        print("Ошибка: укажите название файла")
        sys.exit(1)
    else:
        for i in range(1, len(sys.argv)):
            if sys.argv[i] == '-v':
                print("Полина Александровна Беленькая, гр. N3149\nВариант 2-4-18-3")
                sys.exit(0)
            elif sys.argv[i] == '-n':
                if i + 1 < len(sys.argv):
                    try:
                        n = int(sys.argv[i + 1])
                        i += 1
                    except ValueError:
                        print("Ошибка: введите число после -n!")
                        sys.exit(1)
                else:
                    print("Ошибка: введите число после -n!")
                    sys.exit(1)
            else:
                f_name = sys.argv[i]

    if f_name is not None:
        try:
            with open(f_name, 'wb') as fin:  # Используем контекстный менеджер
                if n == -1:
                    n = random.randint(10, 1000)
                gener(fin, n)
```

```
        except IOError:
            print("Ошибка: данные не записаны в файл")
            sys.exit(1)
    else:
        print("Ошибка: укажите название файла")
        sys.exit(1)


if __name__ == '__main__':
    main()
```